Video Transcript

Video 6: Implementing the Quality Matrix in Python (09:20)

So, let's look at the code for reinforcement learning and solving the problem of teaching the robot to find its way around these set of rooms. We saw that we could represent the rooms in a more compact graphical format here and you set up the reward matrix so that the robot would only move along the paths where there was a one. So, if we're in state one, we can only move to three or to five. So, state one, this is the move to five to our goal, and this would be the move to state three. And, we saw how to develop a Q-matrix that would have our state vertically and the actions that we could take along the horizontal.

So, this was a six-by-six matrix. And we saw how to develop the actual values that we've been needed in here to find our way around, and we were using Bellman's equation to do that. And we saw that the main algorithm to create the Q-matrix would run something like this, that we'd be iterating around a loop. So, we might go for 1000 iterations so that we could develop a stable Q-matrix one that had converged. We choose a random state, we find a list of possible actions, we choose one of the actions. And so that becomes our next row, if you like that we're going to go to. We scan that row for its any Q values in it, and then we update the Q according to this. We look at the reward matrix first, add any rewards in, and then we take gamma times the maximum reward for. In this case, if we were moving to row one or state one, it would be the maximum Q in that row. Okay, so let's go take a look at actual code. I'm going to show you the code in Visual Studio. I'm going to give it you in a Colab notebook, but I suggest you load it into Visual Studio, so you can run it in debug mode and debug it.

So, let's take a look at the code. So, here we are at the beginning. We need NumPy and we're setting up a problem with six dimensions. So, here's our rooms zero, one, two, three, four, five, and we define actions. And, in fact, we only need zero through five of those actions. And we set up a reward matrix. And this we do by hand. This represents the geometry. And remember, and to some extent we, it doesn't have to be 100 for the reward of the goal, you can choose whatever you like.

© MIT xPRO® 2024. All rights reserved.

Some people use 999. So, now we choose a gamma, a discount factor, and we're going to choose a learning rate as well.

I've left it as one here, but it's usually less than one. And this is the function that does all the work, get optimal route. So, this we input a start location, and an end location, and we get the rewards matrix. We choose an ending state and here we set the ending state to have a reward of 100. Now, we're going to calculate the Q-matrix, it's six-by-six in our case. And now we go through the Q learning process. We have our for loop. We choose the current state, so we're going to choose a room to be in, and we choose it at random. We find the playable actions.

What that means is what other rooms can we move to if we're in this room. And we do that by checking whether the reward of moving from the current state to the next state is greater than zero. So, that's why we put ones in our matrix, our reward matrix. And even if when we put the 100, it's greater than zero, so that would be fine. So, we pick a random action, out of the playable actions, we choose one. Now, this does all the work. This is called temporal difference, but it's basically the Bellman equation. Here you see the gamma, we have here that we get the next state.

So, that's now we're picking out a row of where we're going to. And now, we look along that row for the maximum, we choose the maximum argument. So, we're choosing the maximum Q value. So, we get that maximum Q value, we subtract off whatever current, the Q value of the current state next state is. So. if we've got a value already, we're taking that away from whatever we calculate, but it gets added in here in this update, we're going to take alpha. So, alpha is another learning rate. We've got the gamma one and we've got alpha now is a learning rate, we take some fraction of the temporal difference. And it's just because as we loop through this Q-matrix, we'll get new values added. So, the same calculation that we've already done might actually give a different value later. We saw that our maximum reward was going up from 100, and eventually it went up to 200. So, this does all the work, and then we keep looping around to that for however many iterations. I suggest you start with five so you can actually step through it.

So, here we'd put five, that's going to control our iterations. We can put a break point. We can run in debug mode. So here, so we run in debug mode and you see we get here and now we can step through. So, the current state, we're choosing five. It's actually we're at the goal where we

© MIT xPRO® 2024. All rights reserved.

want to be, but that's okay. And then we loop over the playable actions. So, the next state we choose here is one. Now, we find out what our reward is. So, here TD is one, it's allowable, we've got some reward. There are no queue states yet, they're all zero. And now, we're going to update five to one with the value of alpha, which is one times one.

Here now, we'll see things printed out, yes. Now, we're going, and we should be at the end here. We've done our loops so we can end it. So, that's how you work through in debug. Why don't you run this? And as an exercise, I want you to change the alpha to 0.9. So, here we'll have an alpha of 0.9, and change gamma to 0.75. And I want you to run it for 1000 steps and print out the matrix, the Q-matrix at the end, the stable Q-matrix. So, that's your exercise. I'll give you the link to where the code is. As I say, I would run this in Visual Studio if you want, though you're perfectly allowed to run it in Colab. Okay, bye for now.

© MIT xPRO® 2024. All rights reserved.